WEB SECURITY LABS

Beginner's Guide to SQL Injection

Understanding, Demonstrating, and Preventing SQLi Vulnerabilities

1. What is SQL Injection (SQLi)?

SQL Injection (SQLi) is a common web security vulnerability that allows an attacker to interfere with the queries an application makes to its database. It occurs when untrusted user input is concatenated directly into a structured database query without proper sanitization or parameterization. This allows the input to break out of the intended data context and enter the command execution context, tricking the SQL interpreter into executing unintended, malicious database commands.

2. How It Works

SQL databases interpret query commands based on syntax markers, such as quotes and semicolons. When an application takes user text and plugs it directly into a query string, the database cannot distinguish between the structure of the query (written by developers) and the input variables (entered by users).

Consider a dynamic search feature where the query is constructed like this:

Vulnerable Query Assembly:

```
query = "SELECT * FROM products WHERE category = '" + userInput + "'"
```

3. Analyzing the Classic Bypass Payload

In a login bypass scenario, an application checks authentication via username and password inputs. If a user provides a payload specifically crafted with database special characters (like quotes), they can re-shape the syntax.

Payload:	`' OR '1'='1`
Username Field:	`admin' OR '1'='1`
Password Field:	*(Can be left empty)*

When the database runs the query, it interprets the single quote (`'`) in the input as the closing boundary of the string. The subsequent characters `OR '1'='1` are evaluated as logical instructions. Since the expression `'1'='1'` is always **TRUE**, the entire search condition simplifies to **TRUE**, bypassing the password validation

requirement and returning the first record in the database—typically the administrator account.

4. Request Flow & Syntax Manipulation

[User Browser]
Enters payload: `admin' OR '1'='1`

↓ (Sends HTTP Request to Web Server)

[Web Application]
Concatenates raw string directly into SQL template

↓ (Sends Insecure Query to SQL Interpreter)

[SQL Database Engine]
Executes: `SELECT * FROM users WHERE user='admin' OR '1'='1' AND pass=''`
Result: Bypasses check because `'1'='1'` is always TRUE.

↓ (Returns Admin Account Data)

[User Dashboard]
User gains unauthorized access!

5. Primary Risks & Impacts

SQL injection is one of the most critical vulnerabilities because it can lead to:

- **Data Exfiltration:** Read confidential data (e.g. passwords, credit card numbers, PII) directly from any table.
- **Data Loss & Destruction:** Execute statements like `DROP TABLE` or modify balances, permissions, and profiles.
- **Authentication Bypass:** Login into arbitrary accounts without knowing their credentials.
- **Host Compromise:** Write arbitrary files to the server filesystem or execute system shell commands in highly permissive environments.

6. Mitigation & Prevention Methods

Preventing SQL Injection requires one absolute rule: **Never trust user input, and keep query instructions separate from data variables.**

INSECURE (String Concatenation)

```
# Vulnerable Python SQL
cursor.execute(
"SELECT * FROM users WHERE "
f"user = '{username}'"
)
```

SECURE (Parameterized Query)

```
# Secure Parameterization
cursor.execute(
"SELECT * FROM users WHERE "
"user = %s",
(username,)
)
```

Core Defenses:

1. **Prepared Statements (Parameterized Queries):** Forces the database to compile the query outline first, then inserts parameters. Inputs are guaranteed to remain as literal data, never executable instructions.
2. **Input Validation & Sanitization:** Allowlist safe inputs (e.g. ensure numerical fields contain only numbers).
3. **Least Privilege:** Ensure database accounts used by applications only have read/write access to necessary tables and have system file operations disabled.

EDUCATIONAL DISCLAIMER & NOTICE

This document and the associated sandbox labs are designed solely for safe, localized cybersecurity education and defensive verification. Never test, scan, or execute payloads against systems or applications you do not own or have written, explicit authorization to assess. Unauthorized penetration testing is illegal under computer fraud statutes worldwide.